HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

ĐỀ TÀI: XÂY DỰNG WEBSITE ĐỌC TRUYỆN TRỰC TUYẾN METRUYEN

Sinh viên thực hiện: Nguyễn Thành Nam

Mã sinh viên: B23DCCN587

Nhóm: 11

Giảng viên hướng dẫn: Kim Ngọc Bách

Hà Nội - 2026

Mục Lục

1. Tổng quan công việc đã thực hiện trong tuần 3

2. Nội dung thực hiện 3

2.1. Chuẩn hóa phụ thuộc và cấu trúc mã nguồn 3

2.2. Thiết kế API client dùng chung (Axios instance + interceptors) 4

2.3. Quản lý trạng thái xác thực (Zustand + persist) 4

2.4. Kết nối UI Đăng nhập/Đăng ký với Backend API 5

2.5. Bảo vệ route theo trạng thái đăng nhập và vai trò 5

2.6. Đồng bộ điều hướng khi logout cưỡng bức 5

3. Kết quả và kiểm thử 6

4. Kết quả đạt được 8

5. Kế hoạch tuần tới 8

1. Tổng quan công việc đã thực hiện trong tuần

Trong tuần này, mục tiêu trọng tâm là hiện thực luồng xác thực người dùng (authentication) sử dụng JWT (Access Token và Refresh Token), đồng thời thiết lập cơ chế bảo vệ route trên SPA để đảm bảo phân tách rõ giữa truy cập công khai (public), truy cập cần đăng nhập (private) và truy cập quản trị (admin).

Các mục tiêu cụ thể:

- Kết nối form Đăng nhập/Đăng ký với Backend API.
- Thiết kế cơ chế phiên làm việc bằng JWT gồm Access Token và Refresh Token.
- Tự động làm mới Access Token khi hết hạn (interceptor + retry request).
- Bảo vệ route theo trạng thái đăng nhập và vai trò người dùng.

2. Nội dung thực hiện

2.1. Chuẩn hóa phụ thuộc và cấu trúc mã nguồn

- Frontend sử dụng `axios` để xây dựng HTTP client dùng chung và `zustand` để quản lý trạng thái xác thực (auth state).

- Các điểm triển khai chính trong mã nguồn frontend:

```
web/frontend/src/
├── services/api.js                # Axios instance + interceptors
├── store/authStore.js             # Zustand store + persist
├── pages/LoginPage.jsx            # Gọi API /auth/login
├── pages/RegisterPage.jsx         # Gọi API /auth/register
├── components/PrivateRoute.jsx    # Guard route private
├── components/AdminRoute.jsx      # Guard route admin
└── App.jsx                        # Điều hướng khi logout cưỡng bức
```

2.2. Thiết kế API client dùng chung (Axios instance + interceptors)

Tại `services/api.js`, thiết kế API client theo nguyên tắc “một cổng” (single entry) để toàn bộ màn hình dùng cùng cơ chế token:

1. **baseURL:** lấy từ biến môi trường VITE_API_URL

2. **Request interceptor:** Tự động gắn `Authorization: Bearer <accessToken>` nếu tìm thấy `accessToken` trong localStorage.

3. **Response interceptor:**
   - Khi gặp lỗi `401 Unauthorized`, nếu request chưa retry, thực hiện gọi `POST /auth/refresh-token` với `refreshToken`.
   - Nếu refresh thành công, cập nhật `accessToken` mới và retry request ban đầu.
   - Nếu refresh thất bại, gọi `logout()` và phát sự kiện `auth:logout` để ứng dụng điều hướng về trang đăng nhập.

Quy ước lưu trữ token tại client:

- `accessToken`: token truy cập ngắn hạn.
- `refreshToken`: token làm mới phiên.

2.3. Quản lý trạng thái xác thực (Zustand + persist)

Tại `store/authStore.js`, triển khai store theo hướng tối giản nhưng đủ mở rộng:

- State:
  - *`user`:* thông tin người dùng hiện tại (username, email, role, VIP…).
  - *`isAuthenticated`:* trạng thái đăng nhập.

- Hàm nghiệp vụ:
  - *`setAuth(user, accessToken, refreshToken)`*: cập nhật state và lưu token vào localStorage.
  - *`updateUser(user)`:* cập nhật lại thông tin user (dùng cho profile/VIP/M-Point).
  - *`logout()`:* reset state và xóa token.

- Dùng middleware `persist` để lưu bền `user` và *`isAuthenticated`* (key: `auth-storage`), giúp duy trì phiên sau khi reload.

2.4. Kết nối UI Đăng nhập/Đăng ký với Backend API

1. `LoginPage`:

- Gửi `POST /auth/login` với payload `{ emailOrUsername, password }`.
- Nhận `{ user, accessToken, refreshToken }` và gọi `setAuth(...)`.
- Thông báo trạng thái bằng `react-hot-toast`.

2. `RegisterPage`:

- Gửi `POST /auth/register` với payload `{ username, email, password }`.
- Điều hướng về `/login` sau khi đăng ký thành công.

2.5. Bảo vệ route theo trạng thái đăng nhập và vai trò

1. `PrivateRoute`:

- Nếu `isAuthenticated = false` thì điều hướng về `/login`.
- Nếu đã đăng nhập thì render `Outlet`.

2. `AdminRoute`:

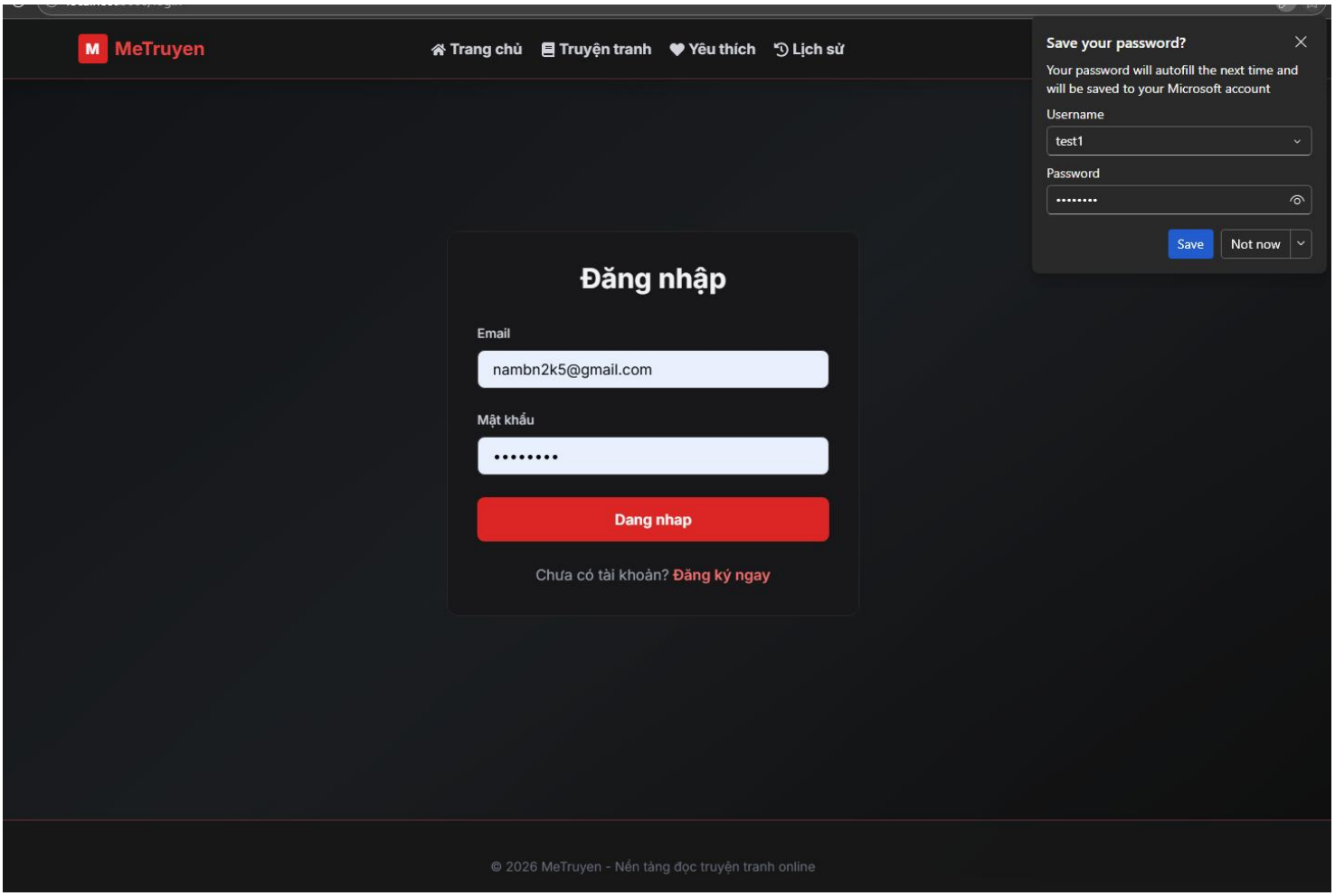
- Yêu cầu `isAuthenticated = true` và `user.role === 'admin'`.
- Nếu không thỏa điều kiện, điều hướng về `/`.

2.6. Đồng bộ điều hướng khi logout cưỡng bức

Trong `App.jsx`, ứng dụng lắng nghe sự kiện `auth:logout` (được phát từ interceptor khi refresh thất bại) để điều hướng về `/login` một cách nhất quán, tránh trạng thái “đăng nhập ảo” khi token hết hạn.

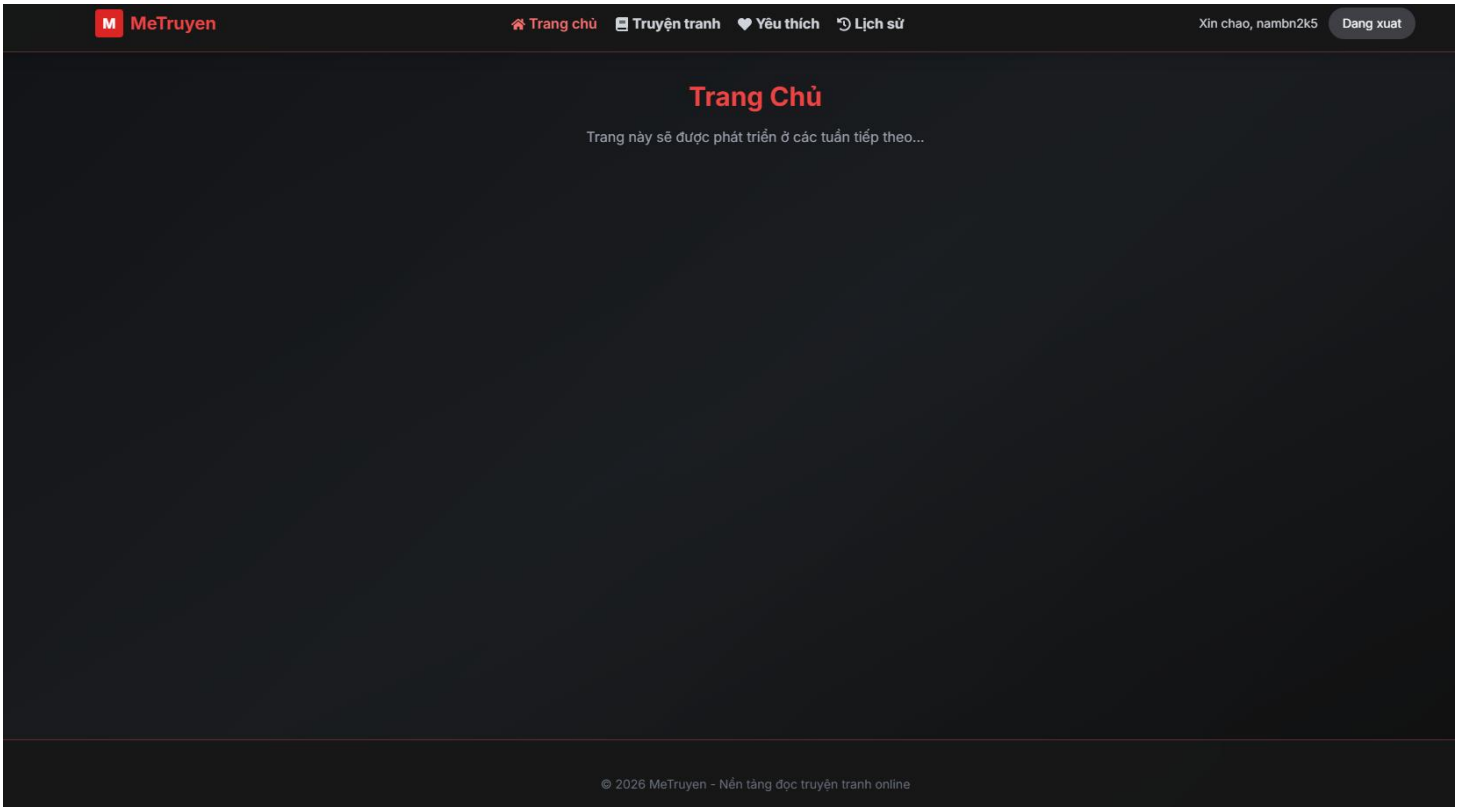
3. Kết quả và kiểm thử

- Đăng ký hợp lệ



=> API trả thành công;
=> UI điều hướng về đăng nhập

- Đăng nhập hợp lệ



=> Lưu token; `isAuthenticated=true`;
=> điều hướng về `/`

- Truy cập route private khi chưa login

=> Điều hướng `/login`

- Truy cập route admin bằng user thường

=> Điều hướng `/`

- Access token hết hạn (401)

=> Tự refresh token và retry request

- Refresh token hết hạn/không hợp lệ

=> Logout cưỡng bức;
=> điều hướng `/login`

4. Kết quả đạt được

- Hoàn thiện luồng xác thực end-to-end: đăng ký/đăng nhập/làm mới token/đăng xuất.
- Thiết lập bảo vệ route theo đăng nhập và vai trò admin.

5. Kế hoạch tuần tới

- Hoàn thiện nhóm trang public: `HomePage`, `ComicsPage`, `ComicDetailPage`.
- Tích hợp API truyện/chương/thể loại.
- Chuẩn hóa trạng thái loading/empty/error cho các màn hình nội dung.